Лабораторна робота 1: короткі теоретичні відомості

Цей конспект допомагає не лише повторити кроки інструкції, а й зрозуміти, що відбувається у проєкті ASP.NET Core Razor Pages. Приклади використовують каталог книг, але так само працюватимуть для товарів, музики, адрес або іншої обраної предметної області.

1. Що створюється в цій лабораторній роботі

Створюється вебзастосунок, у якому дані зберігаються в базі даних і відображаються у браузері. Користувач може виконувати чотири базові операції:

- **Create** — створити запис;
- **Read** — переглянути список або деталі запису;
- **Update** — відредагувати запис;
- **Delete** — видалити запис.

Разом ці операції називають **CRUD**. Наприклад, для каталогу книг запис може містити назву, автора, дату видання, жанр і ціну.

2. ASP.NET Core та Razor Pages

ASP.NET Core — платформа для створення вебзастосунків і вебсервісів мовою C#. Коли користувач відкриває адресу в браузері, застосунок отримує HTTP-запит, виконує потрібний код і повертає HTML-сторінку або іншу відповідь.

Razor Pages — спосіб організації вебсторінок в ASP.NET Core. Кожна сторінка зазвичай має пару файлів:

```
Pages/Books/Index.cshtml       — HTML-розмітка сторінки з Razor-синтаксисом
Pages/Books/Index.cshtml.cs    — C#-код, який готує дані та обробляє дії користувача
```

Наприклад, адреса `/Books` зазвичай відповідає файлу `Pages/Books/Index.cshtml`.

Як обробляється запит

```
Браузер → Razor Page → PageModel → база даних → PageModel → HTML-відповідь → браузер
```

1. Браузер запитує сторінку, наприклад `/Books`.
2. ASP.NET Core знаходить відповідну Razor Page.
3. C#-клас `IndexModel` отримує дані з бази.
4. Файл `.cshtml` підставляє ці дані в HTML.
5. Браузер показує сформовану сторінку.

3. Найважливіші папки та файли

Елемент	Призначення
`Pages/`	Razor Pages: сторінки інтерфейсу та їхні PageModel-класи.
`Pages/Shared/_Layout.cshtml`	Спільний шаблон сайту: меню, заголовок, підвал.
`Pages/_ViewStart.cshtml`	Указує, який макет використовують сторінки.
`Models/`	Класи предметної області: `Book`, `Product`, `Album`.
`Data/`	Контекст EF Core — клас, що описує роботу з БД.
`wwwroot/`	Статичні файли: CSS, JavaScript, зображення.
`appsettings.json`	Налаштування, зокрема рядок підключення до БД.
`Program.cs`	Точка запуску: реєстрація сервісів і налаштування вебзастосунку.

4. Базовий синтаксис C#

Клас і властивості

Клас описує тип об'єктів. Наприклад, клас `Book` описує одну книгу.

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Author { get; set; } = string.Empty;
    public decimal Price { get; set; }
}
```

- `public` означає, що властивість доступна іншим частинам програми.
- `int` — ціле число.
- `string` — текст.
- `decimal` — число для грошей; для цін він точніший, ніж `double`.
- `{ get; set; }` дозволяє прочитати та змінити значення властивості.
- `= string.Empty` створює порожній текст як початкове значення та прибирає попередження про можливе `null`.

`Id` — ідентифікатор запису. Entity Framework Core за домовленістю розпізнає властивість `Id` як первинний ключ таблиці.

Об'єкт

Об'єкт — конкретний екземпляр класу:

```
var book = new Book
{
    Title = "Кобзар",
    Author = "Тарас Шевченко",
    Price = 250.00m
};
```

`var` дозволяє компілятору визначити тип змінної. У цьому прикладі тип `book` — `Book`. Суфікс `m` показує, що число є значенням типу `decimal`.

Умови

```
if (book.Price > 0)
{
    Console.WriteLine("Ціна вказана правильно");
}
else
{
    Console.WriteLine("Ціна має бути більшою за нуль");
}
```

Умова `if` виконує код, якщо вираз у дужках має значення `true`.

Колекції та цикл `foreach`

```
var books = new List<Book>();

foreach (var item in books)
{
    Console.WriteLine(item.Title);
}
```

`List<Book>` — список об'єктів `Book`. `foreach` по черзі бере кожен елемент списку; у цьому прикладі поточний елемент називається `item`.

Методи

Метод — іменований блок коду, який виконує дію.

```
public decimal GetDiscountPrice(decimal discount)
{
    return Price - discount;
}
```

`return` повертає результат роботи методу. Метод вище приймає знижку та повертає ціну зі знижкою.

Асинхронність: `async` і `await`

Робота з базою даних може тривати певний час. Асинхронний код дозволяє вебсерверу не блокувати потік очікуванням відповіді БД.

```
public async Task OnGetAsync()
{
    Books = await _context.Book.ToListAsync();
}
```

- `async` означає, що метод містить асинхронну операцію.
- `await` означає: дочекайся завершення цієї операції, не блокуючи серверний потік.
- `Task` — результат асинхронної операції без окремого значення.
- `ToListAsync()` отримує записи з БД і перетворює їх на список.

Для початку достатньо пам'ятати: якщо метод EF Core закінчується на `Async`, перед ним часто пишуть `await`.

5. Модель і валідація даних

Модель описує, які дані програма зберігає. Щоб не допустити порожню назву або від'ємну ціну, застосовують атрибути валідації.

```
using System.ComponentModel.DataAnnotations;
using Microsoft.EntityFrameworkCore;

public class Book
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Вкажіть назву книги")]
    [StringLength(120)]
    public string Title { get; set; } = string.Empty;

    [Required(ErrorMessage = "Вкажіть автора")]
    [StringLength(100)]
    public string Author { get; set; } = string.Empty;

    [Range(0.01, 1_000_000, ErrorMessage = "Ціна має бути більшою за нуль")]
    [Precision(10, 2)]
    public decimal Price { get; set; }
}
```

- `[Required]` — поле обов'язкове.
- `[StringLength(120)]` — обмеження довжини тексту.
- `[Range(...)]` — допустимий діапазон числового значення.

- `[Precision(10, 2)]` — для ціни дозволено до 10 цифр загалом, з них 2 після коми.

Після зміни моделі потрібно створити та застосувати нову міграцію.

6. Entity Framework Core і база даних

Entity Framework Core (EF Core) — технологія, що дає змогу працювати з таблицями БД через C#-об'єкти. Замість того щоб одразу писати SQL-запит `INSERT INTO Books ...`, можна написати:

```
_context.Book.Add(book);
await _context.SaveChangesAsync();
```

EF Core перетворить цю дію на SQL-команду для обраної бази даних.

`DbContext`

`DbContext` — клас, який представляє сеанс роботи застосунку з базою даних.

```
using Microsoft.EntityFrameworkCore;

public class BookCatalogContext : DbContext
{
    public BookCatalogContext(DbContextOptions<BookCatalogContext> options)
        : base(options)
    {
    }

    public DbSet<Book> Book { get; set; } = default!;
}
```

`DbSet<Book>` приблизно відповідає таблиці книг. Один об'єкт `Book` відповідає одному рядку таблиці.

Реєстрація контексту в `Program.cs`

```
builder.Services.AddDbContext<BookCatalogContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("BookCatalogContext")));
```

Цей код повідомляє застосунку:

1. використовувати клас `BookCatalogContext` для доступу до БД;
2. використовувати SQL Server;
3. взяти рядок підключення з `appsettings.json`.

Рядок підключення

```
"ConnectionStrings": {
  "BookCatalogContext": "Server=
(localdb)\\mssqllocaldb;Database=BookCatalog;Trusted_Connection=True"
}
```

Рядок підключення — це не пароль у звичайному сенсі, а набір параметрів, за якими застосунок знаходить базу даних. Для навчального прикладу LocalDB працює на Windows. У реальному застосунку секрети й паролі не зберігають безпосередньо в `appsettings.json`.

7. Міграції

Міграція — це опис зміни структури бази даних. Якщо створено клас `Book`, EF Core може згенерувати міграцію, яка створить таблицю `Book`.

У Package Manager Console Visual Studio:

```
Add-Migration InitialCreate
Update-Database
```

Альтернатива в терміналі:

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

- `Add-Migration InitialCreate` створює C#-файли міграції.
- `Update-Database` застосовує всі ще не застосовані міграції до БД.

Якщо після зміни властивостей моделі не створити нову міграцію, таблиця в БД не дізнається про ці зміни.

8. Dependency Injection (DI)

Впровадження залежностей означає, що клас не створює важливі об’єкти сам, а отримує їх у конструкторі від ASP.NET Core.

```
public class IndexModel : PageModel
{
    private readonly BookCatalogContext _context;

    public IndexModel(BookCatalogContext context)
    {
        _context = context;
    }
}
```

`BookCatalogContext` передається в `IndexModel` автоматично, тому що його зареєстровано через `AddDbContext` у `Program.cs`. Такий підхід спрощує тестування й керування підключеннями до БД.

9. PageModel і обробники запитів

Файл `Index.cshtml.cs` містить клас, похідний від `PageModel`. Він готує дані для відповідної сторінки `.cshtml`.

```
public class IndexModel : PageModel
{
    public IList<Book> Book { get; set; } = default!;

    public async Task OnGetAsync()
    {
        Book = await _context.Book.ToListAsync();
    }
}
```

- `OnGet` або `OnGetAsync` викликається для звичайного відкриття сторінки — HTTP GET.
- `OnPost` або `OnPostAsync` викликається після надсилання форми — HTTP POST.
- Властивість `Book` передає список із C#-класу до Razor-розмітки.

Приклад збереження форми:

```
[BindProperty]
public Book Book { get; set; } = default!;

public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    _context.Book.Add(Book);
    await _context.SaveChangesAsync();
    return RedirectToPage("./Index");
}
```

`[BindProperty]` дозволяє ASP.NET Core перенести значення з полів HTML-форми у властивості об'єкта `Book`. `ModelState.IsValid` перевіряє правила валідації. Після успішного збереження `RedirectToPage` повертає користувача до списку записів.

10. Razor-синтаксис

Razor дозволяє поєднувати HTML і C# у файлі `.cshtml`. Символ `@` означає перехід до Razor або C#.

Директиви

```
@page
@model BookCatalog.Pages.Books.IndexModel
```

- `@page` робить файл доступною Razor Page, яка може обробити HTTP-запит.
- `@model` указує клас PageModel, з яким працює сторінка.

Виведення значення

```
<h1>@ViewData["Title"]</h1>
<td>@item.Title</td>
```

Razor обчислює вираз після `@` і безпечно виводить його в HTML. Не потрібно вручну перетворювати рядок на HTML.

C#-блок

```
@{
    ViewData["Title"] = "Каталог книг";
}
```

У фігурних дужках можна написати кілька інструкцій C#.

Цикл у Razor

```
@foreach (var item in Model.Book)
{
    <tr>
        <td>@item.Title</td>
        <td>@item.Author</td>
        <td>@item.Price</td>
    </tr>
}
```

Цикл створює один HTML-рядок таблиці для кожної книги. `Model` — це об’єкт класу, указаного в директиві `@model`.

11. Tag Helpers

Tag Helpers — спеціальні атрибути ASP.NET Core у звичайних HTML-тегах. Вони допомагають створювати форми, посилання й повідомлення валідації.

```
<a asp-page="./Edit" asp-route-id="@item.Id">Редагувати</a>
```

Цей код створить посилання на сторінку редагування і передасть ідентифікатор книги, наприклад `/Books/Edit?id=5`.

```
<label asp-for="Book.Title"></label>
<input asp-for="Book.Title" class="form-control" />
<span asp-validation-for="Book.Title" class="text-danger"></span>
```

- `asp-for` пов'язує HTML-елемент із властивістю моделі.
- `label` генерує підпис для поля.
- `input` генерує поле введення з правильним іменем.
- `span asp-validation-for` показує повідомлення, якщо назва не пройшла валідацію.

12. Макет і навігація

`Pages/Shared/_Layout.cshtml` — спільний каркас усіх сторінок. У ньому містяться меню, підключення CSS і підвал.

```
<a class="navbar-brand" asp-page="/Books/Index">Каталог книг</a>
```

Такий запис створює пункт меню. Якщо змінити його, зміна з'явиться на всіх сторінках, що використовують цей макет.

`@RenderBody()` у `_Layout.cshtml` означає місце, куди ASP.NET Core вставляє вміст конкретної Razor Page.

13. Початкові дані (seed data)

Seed-дані — це тестові записи, які автоматично додаються в порожню базу даних. Вони корисні, щоб одразу побачити роботу списку, редагування і видалення.

Важлива перевірка:

```
if (context.Book.Any())
{
    return;
}
```

Вона означає: якщо записи вже є, не додавати дублікати під час кожного запуску застосунку.

14. Типові помилки та перевірка

Симптом	Ймовірна причина	Що перевірити
`Cannot open database`	Не створено БД або не виконано міграцію	Виконати `Update-Database`; перевірити рядок підключення.
`Add-Migration` не знайдено	Немає EF Core Tools або обрано не той проєкт у PMC	Установити пакет Tools; перевірити Default project.
Нове поле не з'явилося в БД	Змінено клас моделі, але не застосовано міграцію	Створити нову міграцію та виконати `Update-Database`.
Дані не зберігаються	Валідація не пройдена або форма не надсилається	Перевірити `ModelState.IsValid` і повідомлення біля полів.
Сторінка `/Books` не відкривається	Неправильний шлях, немає `@page` або не викликано `MapRazorPages()`	Перевірити назву й розташування файлу, `Program.cs`.
Посилання веде не туди	Неправильно вказано `asp-page`	Звірити шлях зі структурою папки `Pages`.

15. Мінімальний чекліст перед здаванням

- [ ] Обрано власну предметну область, а не скопійовано каталог фільмів.
- [ ] Є модель із щонайменше чотирма змістовними полями.
- [ ] Створено і застосовано міграцію.
- [ ] Вебзастосунок показує записи з БД.
- [ ] Працюють створення, перегляд, редагування та видалення.
- [ ] Налаштовано назву сайту й посилання на каталог у меню.
- [ ] Додано щонайменше кілька початкових або власноруч створених записів.
- [ ] Щонайменше два поля мають валідацію.
- [ ] Ви можете пояснити призначення `Model`, `DbContext`, `OnGetAsync`, `OnPostAsync`, `@page` і `asp-or`.

16. Головна ідея лабораторної

Не потрібно запам'ятовувати весь код. Важливо розуміти зв'язок:

```
Клас Book → таблиця Book у БД
DbContext → робота з таблицею
PageModel → отримує/зберігає дані
Razor Page (.cshtml) → показує дані й форму
Scaffolding → генерує початкові CRUD-сторінки
Міграція → синхронізує структуру БД із моделлю C#
```

Коли цей зв’язок зрозумілий, можна замінити `Book` на будь-яку іншу сутність і свідомо змінювати застосунок, а не просто повторювати інструкцію.